

Metaheuristic Algorithms and Applications

Program Report II-2

Group 17 Leader / Member: N26140804 張暉俊

1. Overview

This assignment requires applying a metaheuristic algorithm to the feature selection, clustering, and classification of *Early Stage Diabetes Risk Prediction* (UCI dataset 529). Of the two problems offered in the spec, II-1 (ball-and-beam system control) and II-2 (diabetes risk prediction), I chose to implement II-2 because of my own academic and experiential background.

In this program I continue to use the GA and PSO chosen in Program 1, searching for feature subsets in a 16-dimensional binary mask space, with Logistic Regression as the classifier and `KMeans` (`n_clusters = 2`) as the clustering method.

Part I adopts the `train = test resubstitution` protocol required by the spec, and additionally runs an exhaustive sweep over all 65,535 non-empty masks to obtain “the true optimum under the current LR configuration” as a reference anchor. *Part II* adopts 10×5 -fold stratified cross-validation for a total of 50 outer folds, with an inner 3-fold CV driving the fitness computation of GA/PSO; the outer test fold never participates in the search throughout, so as to avoid leakage. All experiments run both algorithms on the same preprocessed data, and the numerical results and charts are presented in the following sections.

2. Implementation and Experiment

2.1 Dataset and Preprocessing

The UCI dataset cited by the spec has 520 rows $\times$ 17 columns (16 attributes + 1 class label), with original classes of 320 Positive / 200 Negative, presenting roughly a 62/38 class imbalance. All subsequent cross-validation splits in this experiment use stratified splitting to preserve this class ratio. The actual range of *Age* in the CSV is 16–90, inconsistent with the 20–65 marked in the Attribute Information section of the spec document. The dataset characteristics table in the same spec writes Number of Instances = 520, which is consistent with the CSV. However, considering that

1. the figure of 520 rows is itself a fact stated by the spec
2. the cited paper¹ also directly uses the version released by UCI

this experiment chooses to keep the original *Age* without truncation, filtering, or other processing, and records this discrepancy here.

For preprocessing, all Yes/No, Male/Female, Positive/Negative categorical columns are encoded as 0/1. *Age* keeps its original integer value and only undergoes z-score standardization. In *Part I*, because `train = test`, the scaler is fit directly on the full 520 rows. In *Part II*, it is strictly fit on the training subset of each outer fold and then applied to the test subset of that fold, so as

to prevent the statistics of the test data from leaking into the search loop. The remaining binary columns are not scaled.

2.2 Genetic Algorithm (GA)

The binary chromosome is a 16-bit encoding, each bit corresponding to whether an attribute is selected. The settings of each operator are summarized as follows:

- Tournament selection, $k = 3$
- Uniform crossover, each bit independently swapped with probability 0.5
- Bit-flip mutation, each bit flipped with probability $1/16$
- Elitism: the best individual of the current generation is preserved into slot 0 of the next generation

The initial population forbids the all-zero mask. During the search, if crossover or mutation produces an empty mask, the fitness layer handles it with $-\infty$. Compared with the real-valued GA I originally used in Program I, this time, because the search space is itself a 16-dimensional binary one, binary encoding is more natural than real-valued, and crossover and mutation also return to their standard forms in the binary domain.

2.3 Particle Swarm Optimization (PSO)

The position is a continuous vector in $[0, 1]^{16}$, converted into a binary mask with 0.5 as the threshold during evaluation; then pbest and gbest are stored as continuous positions, but **best_pos** is always the thresholded mask when reported externally. Each hyperparameter is set as follows:

- The inertia weight w decreases linearly from 0.9 to 0.4
- The cognitive / social coefficients are set to $c_1 = c_2 = 2$
- The velocity clamp is $[-v_{\max}, v_{\max}]$

Among these, if a particle’s continuous position is entirely below 0.5 and becomes an empty mask after thresholding, the fitness layer returns $-\infty$, making that particle unable to become pbest / gbest, which is equivalent to excluding the empty mask from the search space.

2.4 Fitness Function

The fitness used in this program is defined as:

$$\text{fitness}(\text{mask}) = \text{accuracy}(\text{mask}) - 0.005 \times \frac{|\text{selected}|}{16}$$

Here $\lambda = 0.005$ is deliberately set very small, because the entire dynamic range of the penalty is $[0, 0.005]$, and each additional selected feature only adds a penalty of $\lambda/16 \approx 0.0003$. By comparison, the smallest unit of change for *Part I* resubstitution accuracy on 520 rows is $1/520 \approx 0.0019$ —the gain from misclassifying one fewer sample is, in my view, worth about 6 features’ worth of penalty. Therefore, unless two masks have exactly the same accuracy, the penalty will never dominate the

comparison; it only prefers fewer features in the case of a tie. Subsequent experiments also verified that λ does not push the 11-feature global-best mask away from the optimal position.

In terms of implementation, the fitness object is simultaneously responsible for the empty-mask sentinel, the integer-mask cache, and the penalty computation, the three things concentrated within a single `__call__`

```
# lib/fitness.py
@dataclass
class FitnessFunction:
    accuracy_fn: Callable[[np.ndarray], float]
    lambda_penalty: float = 0.005
    n_features: int = N_FEATURES
    cache: dict[int, float] = field(default_factory=dict)
    raw_accuracy: dict[int, float] = field(default_factory=dict)
    n_evaluations: int = 0
    n_cache_hits: int = 0

    def __call__(self, bits: np.ndarray) -> float:
        mask = bits_to_int(bits, self.n_features)
        if mask == 0:
            return float("-inf")
        if mask in self.cache:
            self.n_cache_hits += 1
            return self.cache[mask]
        cols = mask_to_columns(mask, self.n_features)
        acc = float(self.accuracy_fn(cols))
        size_ratio = mask.bit_count() / self.n_features
        score = acc - self.lambda_penalty * size_ratio
        self.cache[mask] = score
        self.raw_accuracy[mask] = acc
        self.n_evaluations += 1
        return score
```

`accuracy_fn` is an injected function with different implementations depending on the protocol:

- *Part I* is “treat the mask as a feature subset, fit-predict on the 520 rows, and compute the resubstitution accuracy”
- *Part II* is “perform inner 3-fold CV on the outer training set and return the mean accuracy”.

This design lets the same fitness/cache/penalty logic be shared across both Parts. The cache is keyed by integer bitmask, because when GA and PSO search in the 16-dimensional binary space, the probability of the same mask recurring is inherently very high.

2.5 Classifier and Clustering

For the classifier, I chose to use Logistic Regression (sklearn `LogisticRegression`) for several reasons:

1. it is precisely one of the classifiers adopted by Islam et al.¹; only by using the same model can we directly compare against that paper’s baseline
2. LR trains extremely fast, which is a necessary condition for the exhaustive sweep that needs to fit 65,535 times, as well as for the thousands of inner evaluations that GA/PSO run within each of the 50 outer folds
3. the linear model introduces no additional feature interactions, keeping the feature-selection signal of the mask itself clean.

Besides that, for the hyperparameters

- the parameter `solver='lbfgs'`
- L2 penalty, `C=1.0`
- `max_iter=1000`
- `fit_intercept=True`
- threshold 0.5
- no class weights.

All of the exhaustive baseline, *Part I*, and *Part II* use this same LR configuration. This also means that the “best mask” found by the exhaustive sweep is “the best under this LR configuration,” not the absolute best; the ranking may change after switching the LR configuration or the classifier.

The clustering method uses `KMeans (n_clusters = 2)`, first fit on the training data, then mapping the two clusters to the Positive / Negative labels by a majority vote on the training data, and finally computing accuracy on the evaluation data. The clustering result is only presented alongside the classification result in the *Part I/II* reports and does not enter the GA/PSO fitness computation—because the spec requires all three of “feature extraction, clustering, and classification,” but does not require incorporating clustering into the optimization objective; separating it instead lets us observe “the gap between the same mask under supervised and unsupervised settings.”

2.6 Baselines and Parameter Settings

To give the GA/PSO results a basis for comparison, this experiment prepared five baselines:

- **All-features Logistic Regression** (mask = `0xFFFF`): the control group without feature selection, the lower bound that GA/PSO must surpass.
- **All-features Random Forest** (sklearn default parameters, with fixed `random_state=0`): a strong-classifier reference that overfits to 1.0000 under resubstitution, used to show that “with the same data, zero error is achievable if the model is unconstrained.”
- **Majority-class baseline**: always predicts Positive, accuracy = 320/520, the absolute lower bound of the classification problem.
- **Exhaustive LR sweep**: enumerates all 65,535 non-empty 16-bit masks, recording the resub-

stitution accuracy of each mask. The tiebreak rule here is accuracy descending, n_features ascending, mask ascending, and under this rule the true best mask used for *Part I* comparison is selected.

- **Islam et al.¹**: that paper’s LR achieves an accuracy of 0.9240 under 10-fold CV, referenced for *Part II* comparison.

All parameters of the experiment are organized in the table below:

Parameter	GA	PSO	Shared
Population size	50	50	—
Iterations	100	100	—
Selection / update	Tournament $k = 3$	$w: 0.9 \rightarrow 0.4, c_1 = c_2 = 2$	—
Crossover	Uniform (per-bit 0.5)	—	—
Mutation	Bit-flip (per-bit 1/16)	—	—
Elitism	Slot 0 preserved	—	—
Representation	Binary 16-bit	$[0, 1]^{16} \rightarrow \text{threshold } 0.5$	—
λ (n_features penalty)	—	—	0.005
Classifier	—	—	LR (lbfgs, L2, C = 1.0, max_iter = 1000)
Cluster	—	—	KMeans (k = 2)

Table 1: Experimental parameter settings

The experimental environment runs on CPU with Python 3.12 + scikit-learn.

For the final running time:

- *Part I* takes about 41 seconds in total (GA 32.06 s, PSO 9.10 s)
- *Part II* takes 179.14 seconds in total.

Among these, the random seed is independently assigned per seed in *Part I*, and independently assigned per fold in *Part II*.

3. Part 1

3.1 Protocol

Following the spec’s requirement, the `train = test` resubstitution protocol is adopted, performing feature selection and classification simultaneously on the full 520 rows, where the model’s prediction on the training data is the test result. GA and PSO each run 30 independent seeds, each run being `pop = 50, max_iter = 100`. After each seed finishes, the best mask, its raw accuracy, and the number of selected features are recorded, and for that seed’s best mask the five spec-required

metrics (Accuracy / Sensitivity / Specificity / Precision / F-measure) are computed on the full data, one set each for classification and **KMeans** clustering. After all 30 seeds finish, the best run is selected for final presentation according to the aforementioned tiebreak rule.

3.2 Results

The best resubstitution accuracy found by the exhaustive sweep under the current LR configuration is 0.9500, a value jointly achieved by mask 12239 (11 features) and mask 61135 (12 features). According to the tiebreak rule, mask 12239 is reported preferentially, whose feature combination is *Age, Gender, Polyuria, Polydipsia, Polyphagia, Genital thrush, visual blurring, Itching, Irritability, delayed healing, muscle stiffness*.

All 30 GA seeds converge to mask 12239, with raw accuracy 30/30 at 0.9500. PSO reaches 0.9500 in 9/30, with the remaining 21 seeds falling between 0.9365–0.9481, raw accuracy averaging 0.9451 ± 0.0045 . It is worth noting that those 9 PSO seeds reaching 0.9500 all hit the two global optima enumerated by the exhaustive sweep (mask 12239 for 4 seeds, mask 61135 for 5 seeds), without exception, and none discovered a new 0.9500 mask outside the exhaustive set. The best runs of both GA and PSO fall on mask 12239 by the tiebreak rule, so the subsequent five metrics are completely identical.

Metric	Classification	Clustering (KMeans)
Accuracy	0.9500	0.6154
Sensitivity	0.9500	1.0000
Specificity	0.9500	0.0000
Precision	0.9682	0.6154
F-measure	0.9590	0.7619

Table 2: *Part I* best-mask classification and clustering metrics (the best runs of both GA and PSO fall on mask 12239)

To delimit where this 0.9500 sits, the table below lists several reference points for *Part I*:

Reference	Accuracy
Exhaustive best (mask 12239 / 61135 jointly)	0.9500
All-features Logistic Regression	0.9308
All-features Random Forest	1.0000
Majority-class baseline	0.6154

Table 3: *Part I* reference data

3.3 Observations

The most direct observation of *Part I* is that GA fully converges to the global optimum. As can be seen in the figure below, GA’s standard-deviation band almost disappears into a single line after the 30th iteration. Evidently, in the 16-dimensional mask space (search space $2^{16} \approx 65k$), a GA with $\text{pop} = 50$ and $\text{iter} = 100$ is clearly more than sufficient at this scale; the selection pressure provided by elitism and tournament $k = 3$ is enough to lock onto the global optimum within ~ 40 generations.

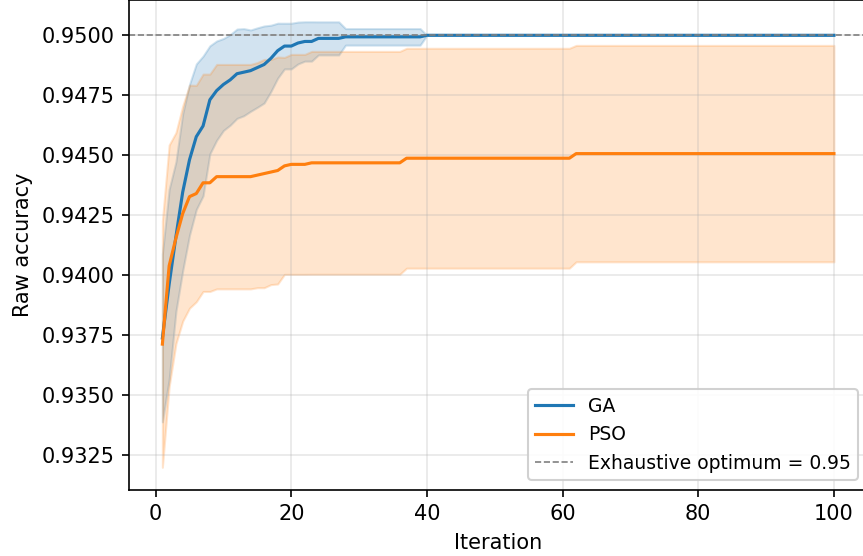


Figure 1: *Part I* raw accuracy convergence curves, mean ± 1 standard deviation over 30 seeds. The dashed line is the exhaustive optimum 0.9500.

In contrast to GA’s full attendance, PSO is quite interesting. The seeds hitting the global optimum are a minority, while the rest get stuck in local optima, which is a typical failure mode of PSO in a discrete search space. Because the position is continuous and the threshold is a hard cut, once the swarm’s gbest corresponds to a certain mask, the perturbations of individual particles in its neighborhood mostly produce the same mask afterward.

This can also be seen from the cache hit rate being as high as 91.7% (GA is 68.6%). What is worth reconsidering is that, among that batch of PSO seeds reaching 0.9500, some fall on mask 61135 rather than the tiebreak-preferred 12239. I find myself thinking that perhaps PSO’s path to success does not necessarily fall on the same point GA converges to, yet the points both fall on are the true optima marked out by the exhaustive sweep, with no “anomalous solutions.” From this contrast, one can infer that the PSO seeds that fell short were not finding a “different but equally good” solution, but genuinely failed to converge.

The figure below allows a more precise reading: GA’s selection frequency falls exactly on the 11 features of mask 12239, while PSO, because some seeds fall on 61135 or other local optima, scatters

across a number of features.

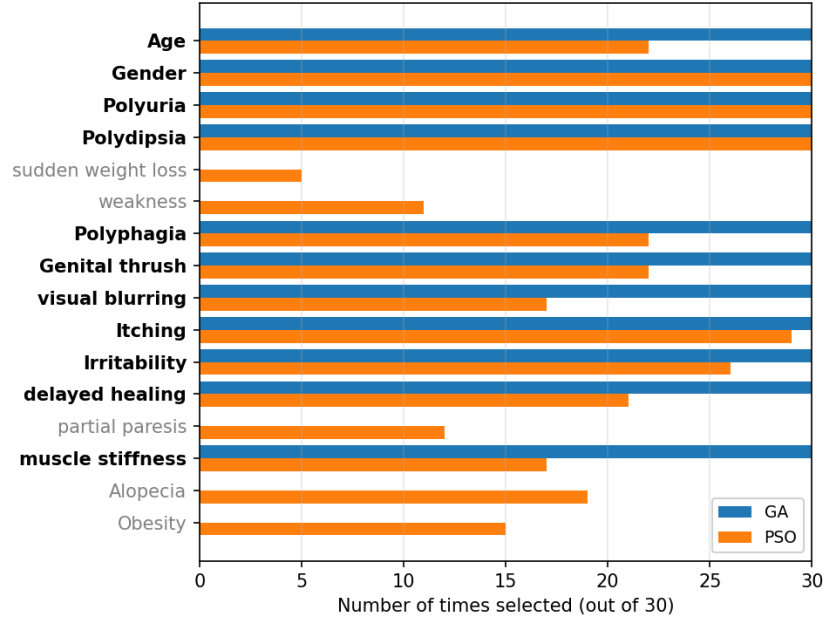


Figure 2: *Part I* the number of times each attribute is selected by the best mask (max = 30). The bold y-axis labels are the 11 features contained in the exhaustive best mask 12239.

It can be seen that the clustering accuracy of 0.6154 happens to equal the majority-class baseline, a negative result deliberately retained. Under a purely unsupervised setting, for a feature space composed of 11 mostly-binary features plus one standardized *Age*, the two clusters found by *KMeans* have almost no correspondence with the true Positive/Negative labels. In other words, the two clusters of *KMeans* happen to be mapped by majority vote in a way that “assigns everything to Positive,” equivalent to the behavior of the majority baseline. This point is more directly visible when spread out into the five metrics

1. sensitivity 1.0
2. specificity 0.0
3. precision 0.6154
4. F-measure 0.7619
5. sensitivity at full marks while specificity is zero

These are exactly what “guessing all Positive” looks like in metrics. This phenomenon is slightly different under the CV setting of *Part II*.

4. Part 2

4.1 Protocol

Following the spec’s requirement, 10 repetitions of 5-fold stratified cross-validation are performed, for a total of 50 outer folds. The core design of the entire process is that “the outer test does not

participate in any model-selection decision,”avoiding the selection bias of the feature-selection stage from mixing into the test accuracy. The loop skeleton of the outermost `main()` is responsible for splitting the 50 outer folds and for the matter of “re-fitting the *Age* scaler per fold”:

```
# run_part2.py: main()
# ... dataset load omitted
outer = RepeatedStratifiedKFold(
    n_splits=N_FOLDS, n_repeats=N_REPEATS, random_state=0
)
# ... timer / folds list setup omitted

for fold_i, (tr_idx, te_idx) in enumerate(outer.split(X_raw, y)):
    X_tr_raw, X_te_raw = X_raw[tr_idx], X_raw[te_idx]
    y_tr, y_te = y[tr_idx], y[te_idx]
    mean, std = fit_age_scaler(X_tr_raw)
    X_tr = apply_age_scaler(X_tr_raw, mean, std)
    X_te = apply_age_scaler(X_te_raw, mean, std)

    fold_out = run_one_fold(X_tr, y_tr, X_te, y_te, fold_seed=fold_i)
# ... progress print / aggregation omitted
```

`run_one_fold` internally runs GA and PSO once each; after each search finishes, it immediately re-fits LR on the outer training set using the best mask and evaluates the outer test:

```
def run_one_fold(X_tr, y_tr, X_te, y_te, fold_seed):
    fold = {}
    for name, algo in (("ga", ga_maximize), ("pso", pso_maximize)):
        f = make_cv_fitness(X_tr, y_tr, seed=fold_seed)
        r = algo(f, pop_size=POP_SIZE, max_iter=MAX_ITER, seed=fold_seed)
        mask = bits_to_int(r["best_pos"])
        # ... convergence bookkeeping omitted
        train_m, test_m, clust_tr, clust_te = evaluate_mask(
            mask, X_tr, y_tr, X_te, y_te
        )
        fold[name] = {
            "mask": int(mask),
            # ... other fields omitted
        }
    return fold
```

What truly isolates the outer test lies in `make_cv_fitness`. It splits the outer training set once more into a 3-fold StratifiedKFold; all GA/PSO fitness computations look only at this inner split, and the outer test never enters the fitness evaluation:

```

def make_cv_fitness(X_train, y_train, seed):
    skf = StratifiedKFold(n_splits=INNER_FOLDS, shuffle=True, random_state=seed)
    splits = list(skf.split(X_train, y_train))

    def accuracy_fn(cols):
        accs = [
            fit_predict_accuracy(
                X_train[tr][:, cols], y_train[tr],
                X_train[te][:, cols], y_train[te],
            )
            for tr, te in splits
        ]
        return float(np.mean(accs))

    return FitnessFunction(accuracy_fn=accuracy_fn, lambda_penalty=LAMBDA)

```

KMeans clustering is also evaluated within `evaluate_mask` using the same best mask, computing one set of five metrics each on the outer training and test sets. GA and PSO each run independently once per outer fold (50 + 50 searches in total), with the seed independently assigned per fold.

4.2 Results

The entire 50-outer-fold process elapsed 179.14 seconds. The training and testing metrics of GA and PSO are listed separately below, all being the mean $\pm$ std over 50 folds.

Metric	Train	Test
Accuracy	0.9380 $\pm$ 0.0091	0.9177 $\pm$ 0.0267
Sensitivity	0.9429 $\pm$ 0.0100	0.9275 $\pm$ 0.0359
Specificity	0.9301 $\pm$ 0.0203	0.9020 $\pm$ 0.0487
Precision	0.9559 $\pm$ 0.0122	0.9390 $\pm$ 0.0280
F-measure	0.9493 $\pm$ 0.0073	0.9326 $\pm$ 0.0223
n_features	—	10.12 $\pm$ 1.83

Table 4: *Part II* GA results (50 outer folds, mean $\pm$ std)

Metric	Train	Test
Accuracy	0.9356 $\pm$ 0.0101	0.9160 $\pm$ 0.0269
Sensitivity	0.9420 $\pm$ 0.0092	0.9272 $\pm$ 0.0333
Specificity	0.9254 $\pm$ 0.0206	0.8980 $\pm$ 0.0458
Precision	0.9530 $\pm$ 0.0124	0.9364 $\pm$ 0.0268
F-measure	0.9474 $\pm$ 0.0081	0.9313 $\pm$ 0.0223
n_features	—	10.48 $\pm$ 1.88

Table 5: *Part II* PSO results (50 outer folds, mean $\pm$ std)

The five metrics of clustering (KMeans) are listed separately in the table below, likewise on both the training and testing sides.

Metric	GA Train	GA Test	PSO Train	PSO Test
Accuracy	0.6946 $\pm$ 0.0880	0.6900 $\pm$ 0.0874	0.6987 $\pm$ 0.0907	0.7006 $\pm$ 0.0902
Sensitivity	0.8536 $\pm$ 0.1429	0.8503 $\pm$ 0.1524	0.8487 $\pm$ 0.1432	0.8559 $\pm$ 0.1381
Specificity	0.4402 $\pm$ 0.4338	0.4335 $\pm$ 0.4284	0.4586 $\pm$ 0.4334	0.4520 $\pm$ 0.4313
Precision	0.7583 $\pm$ 0.1518	0.7533 $\pm$ 0.1481	0.7638 $\pm$ 0.1510	0.7631 $\pm$ 0.1517
F-measure	0.7782 $\pm$ 0.0380	0.7735 $\pm$ 0.0456	0.7796 $\pm$ 0.0428	0.7825 $\pm$ 0.0392

Table 6: *Part II* clustering (KMeans) five metrics (50 outer folds, mean $\pm$ std)

Reference	Test accuracy
Islam et al. ¹ , LR, 10-fold CV	0.9240
GA test accuracy (this assignment)	0.9177 $\pm$ 0.0267
PSO test accuracy (this assignment)	0.9160 $\pm$ 0.0269

Table 7: *Part II* reference baselines

The range of GA test accuracy over the 50 folds is [0.8558, 0.9712], and PSO is likewise [0.8558, 0.9712], with their interquartile ranges overlapping. Among the 50 folds, GA produced 48 distinct best masks in total and PSO 49, with almost every fold selecting a different mask.

4.3 Observations

The test accuracy of *Part II* indeed falls in the same interval as the LR 10-fold CV result of Islam et al.¹ (0.9240), and both are slightly lower than the value in that study¹. The gap is reasonable. Among the reasons, in program2 we chose the stricter split of 50 outer folds and re-performed feature selection in every fold; relative to that study¹, which uses only all 16 features with 10-fold CV, the expected value of our process’s test is inherently pulled down and its std raised by fold-split

variance. The train $\rightarrow$ test gap is about 2 percentage points (GA 0.9380 $\rightarrow$ 0.9177, PSO 0.9356 $\rightarrow$ 0.9160), a healthy generalization gap with no obvious sign of overfitting. The box plot below also shows that the outer test accuracy of both GA and PSO clusters densely around the 0.9240 baseline of that study¹.

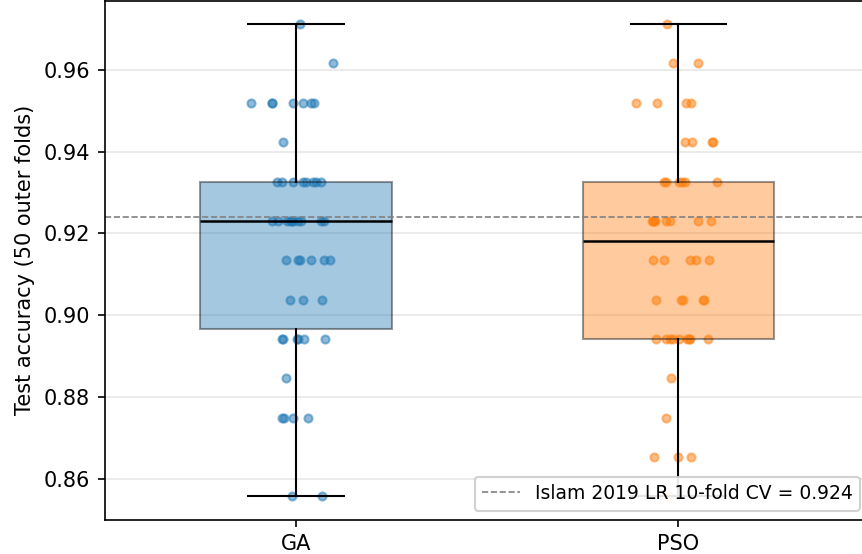


Figure 3: *Part II* outer test accuracy distribution. The dashed line is the LR 10-fold CV baseline 0.9240 of Islam et al.¹

The gap between GA and PSO in *Part II* is small, but PSO’s `n_features` averages 0.36 more than GA (10.48 vs 10.12), and its specificity is slightly lower (0.8980 vs 0.9020). However, this “PSO selecting a few more features” is not a tendency stable across Parts; *Part I*’s PSO average feature count of 10.93 is in fact slightly fewer than GA’s 11.00, so this is more like an incidental difference under the *Part II* CV setting rather than an algorithmic essence. The convergence curves in the figure below also show that the two are almost coincident on the inner CV, with the standard-deviation band remaining persistently wide, reflecting that the inner-CV optimum of each fold is inherently different.

The clustering accuracy in *Part II* is, on the contrary, noticeably higher than the 0.6154 of *Part I* (GA 0.6900, PSO 0.7006), because the outer test and training of *Part II* are not the same data, giving `KMeans` more opportunity across folds to map clusters to the true labels. But even so, clustering is still far below classification, again confirming that `KMeans` struggles to separate Positive from Negative on this semi-binary feature space relying on unsupervised structure alone.

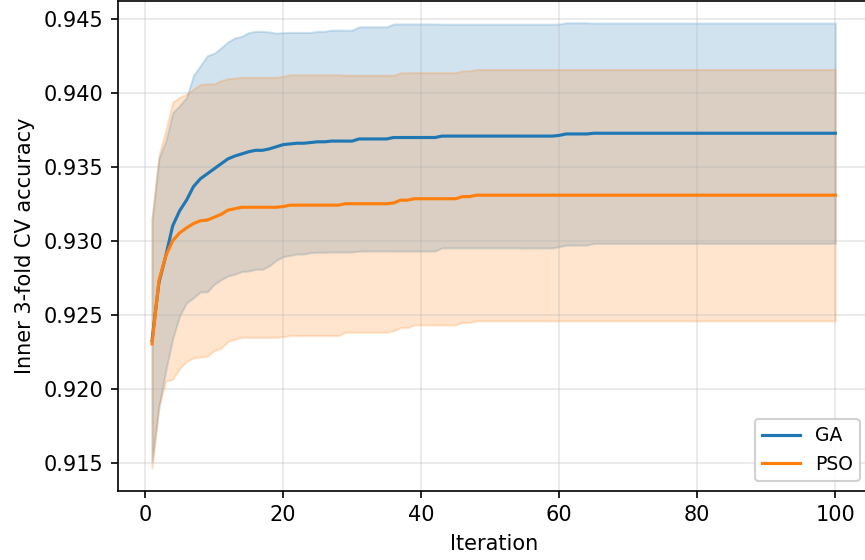


Figure 4: *Part II* inner 3-fold CV accuracy convergence curves, mean $\pm$ 1 standard deviation over 50 folds.

It can be found that, according to the five metrics of clustering, *Part II* also surfaces a phenomenon not visible in *Part I*: the cross-fold variance of specificity is abnormally large (GA 0.4335 ± 0.4284 , PSO 0.4520 ± 0.4313), the standard deviation being almost as large as the mean. This means that *KMeans*’s recognition of the negative class is sometimes good and sometimes bad across folds; in good folds the two clusters happen to match the true labels and specificity is pulled very high, while in bad folds it collapses back to the all-Positive-guessing state of *Part I*. The $0.69 / 0.70$ of accuracy smooths over this bimodal distribution, which is also where looking at the five metrics together is more informative than looking at accuracy alone.

But the most noteworthy is the gap shown in the figure below between “the *Part I* best feature set” and “the *Part II* CV-stable feature set.”

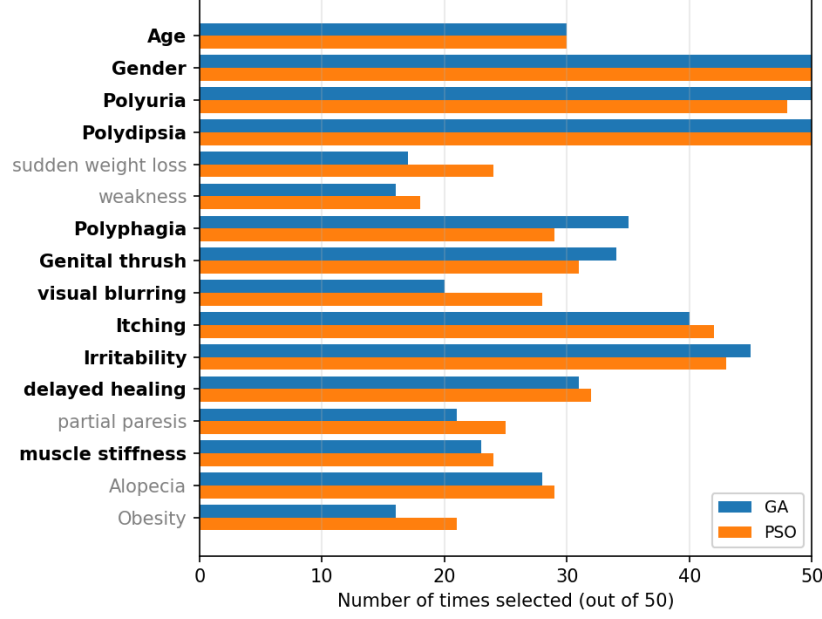


Figure 5: *Part II* the number of times each attribute is selected (max = 50). The bold y-axis labels are the 11 features contained in the *Part I* exhaustive best mask 12239; one can see that there is a gap between the *Part I* “true best” feature set and the *Part II* “CV-stable” feature set.

Comparing the best 11 features identified by *Part I* exhaustive with the selection frequency over *Part II*’s 50 folds, they can be divided into three groups:

- **The core that both algorithms are extremely stable on in *Part II*:** *Gender* (GA/PSO 50/50 each), *Polydipsia* (50/50), *Polyuria* (GA 50/50, PSO 48/50), all uncontroversial core features.
- **Mandatory in *Part I*, drifting in *Part II*:** *Age* (30/50), *visual blurring* (GA 20, PSO 28). Features that are mandatory under resubstitution are not necessarily the most effective in every fold under CV.
- **Never selected in *Part I*, yet common in *Part II*:** *Alopecia* (28–29/50), *partial paresis* (21–25/50), *sudden weight loss* (17–24/50). These “redundant in *Part I*” features provide stability under CV.

The meaning of these groupings and comparisons is that *Part I*’s “global-best mask” is a result overfitted to “one specific set of 520 rows”; when the data split changes and what is evaluated is generalization ability, the originally seemingly-redundant features instead provide stability, while the originally-mandatory features are not necessarily the most effective in every fold.

5. Questions

5.1 Do you have to use all the attributes?

After completing the above experiments, the verification is clear that you do not. Because under this dataset and this program’s LR configuration, the resubstitution accuracy using all 16 attributes is 0.9308, but the resubstitution accuracy using only 11 attributes (mask 12239) is 0.9500. This shows that fewer features instead give a higher accuracy, a gap of about 2 percentage points.

The CV results of *Part II* also support this conclusion. GA selects an average of 10.12 features over the 50 folds, with test accuracy 0.9177. PSO selects an average of 10.48 features, with test accuracy 0.9160. Both significantly reduce the number of features below all-features, while still maintaining a test accuracy at the same level as the baseline reported by Islam et al.¹.

More specifically, the five features *sudden weight loss* / *weakness* / *partial paresis* / *Alopecia* / *Obesity* are all unselected in the best mask 12239 identified by *Part I* exhaustive. However, among them *Alopecia* and *Obesity* actually appear in another jointly-best mask 61135 (which uses these two to replace *visual blurring*, likewise reaching 0.9500), showing that they are not useless but interchangeable with other features. The only ones truly absent from both global-best masks are *sudden weight loss* / *weakness* / *partial paresis*; these three are the ones that truly provide no positive help to LR’s Positive/Negative classification on this data, and may even introduce noise. From a clinical perspective, this result does not negate the medical significance of these symptoms, but rather points out that, under the condition that all 16 signs are simultaneously available, 11 principal features are sufficient to carry the vast majority of the classification information.

5.2 Is it possible to find the best combination of attributes to make the highest classification Accuracy rates?

In this assignment, the operational definition of “best” is “the mask that maximizes resubstitution accuracy under a fixed LR configuration (lbfgs, L2, C = 1.0, max_iter = 1000, threshold 0.5).” Under this definition, the exhaustive sweep of 65,535 non-empty masks can be completed within ~80 seconds, and clearly finds the two jointly-best masks with accuracy = 0.9500 (12239 and 61135). GA converges to one of them in 30/30 under 30 independent seeds, PSO hits in 9/30, and the success of both is consistent with the exhaustive result, with no better solution found outside the exhaustive set. In other words, at this scale of 16 attributes, metaheuristic search can find the global optimum, and the exhaustive sweep further guarantees the existence of this global optimum. So it can be found, but consensus needs to be reached on the definition of “best,” because “best” is something that changes with the evaluation protocol and the classifier configuration.

The CV results of *Part II* show that the best mask selected by each outer fold is almost always different (GA 48/50 kinds, PSO 49/50 kinds), which means that a single mask “best for all folds” is unrealistic, because the statistical distribution of the training data differs across folds, and their respective optimal feature combinations are inherently different. So if we change the definition of “best” to “generalizes best to unseen data,” then what exists is not a single best mask, but a group

of high-frequency stable features plus a set of replaceable secondary features. The former is represented by *Gender*, *Polyuria*, *Polydipsia*, *Irritability*, *Itching*, the five of which have a selection rate of 40+/50 in *Part II* for both the GA and PSO algorithms. Changing the LR configuration, the classifier, or the evaluation protocol, the specific members of the “best combination” will drift accordingly. So the conclusion is that, under a fixed configuration, a single dataset, and resubstitution evaluation, the best combination exists and can be found by metaheuristics. Relaxed to the generalization sense, “best” should be understood more as “a high-frequency stable subset plus several interchangeable secondary features” rather than a single answer.

6. References

- [1] Islam, MM Faniqul, et al. “Likelihood prediction of diabetes at early stage using data mining techniques.” *Computer Vision and Machine Intelligence in Medical Image Analysis: International Symposium, ISCMM 2019*. Singapore: Springer Singapore, 2019.